IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Kardelen Geçkin

November 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Data collected from SpaceX API, public launch tables and course datasets.
- Analysis used Python, SQL, Folium, Plotly Dash and four classifiers.
- All four models scored 83.3% on a small 18-record test set.

Introduction

- Falcon 9 first-stage recovery is a key enabler of reusable launch operations.
- Research question: Can historical launch attributes predict landing success?
- Investigate launch site, orbit, payload, flight number and booster characteristics.

Section 1

Methodology



Methodology

Data collection: SpaceX API and Wikipedia tables.

- Preparation: missing values, landing outcomes and binary Class label.
- Exploration: Python charts and SQL queries.
- Interactivity: Folium location maps and Dash filters.
- Prediction: tuned Logistic Regression, SVM, Decision Tree and KNN.

Data Collection

SpaceX REST endpoints and historical launch tables.

IBM coursework CSV files for SQL, mapping and dashboard exercises.

The notebooks use related but not always identical datasets.

Data Collection – SpaceX API

- REST requests return launch records as JSON.
- Resolve rocket, payload and launchpad attributes.
- Normalize fields into Pandas tables.

Notebook evidence · SpaceX API

Code excerpt

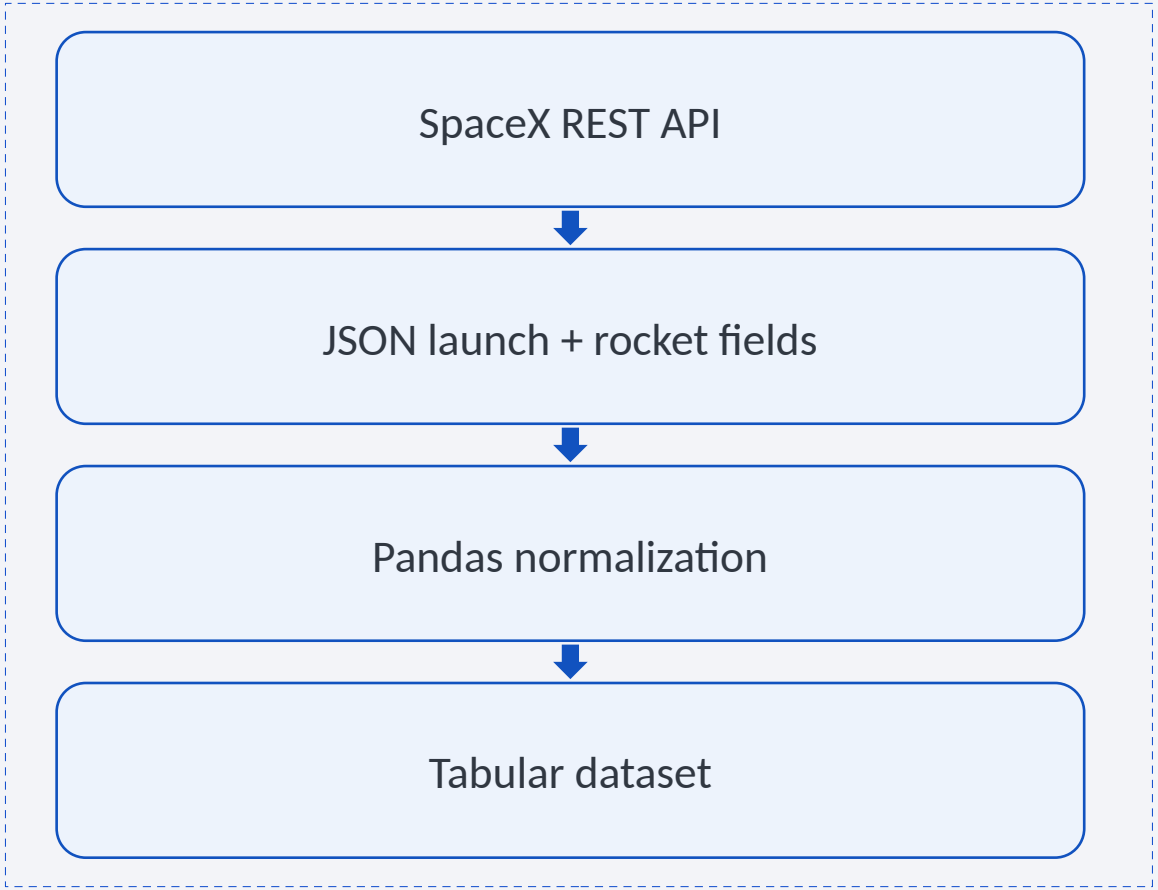
```
response = requests.get(static_json_url)
data = pd.json_normalize(response.json())
data = data[['rocket','payloads','launchpad','cores',
             'flight_number','date_utc']]
getBoosterVersion(data)
getLaunchSite(data)
df = pd.DataFrame(launch_dict)
```

Observed output / evidence

```
response.status_code  ->  200
BoosterVersion[0:5] -> ['Falcon 1', 'Falcon 1', ...]
data_falcon9.to_csv("dataset_part_1.csv", index=False)
```

GitHub notebook: Module 1 · SpaceX API collection

[GitHub notebook link]



Data Collection - Scraping

- Read Wikipedia Falcon 9 tables.
- Parse rows with BeautifulSoup.
- Clean headers and export structured CSV.

Notebook evidence · Web scraping

Code excerpt

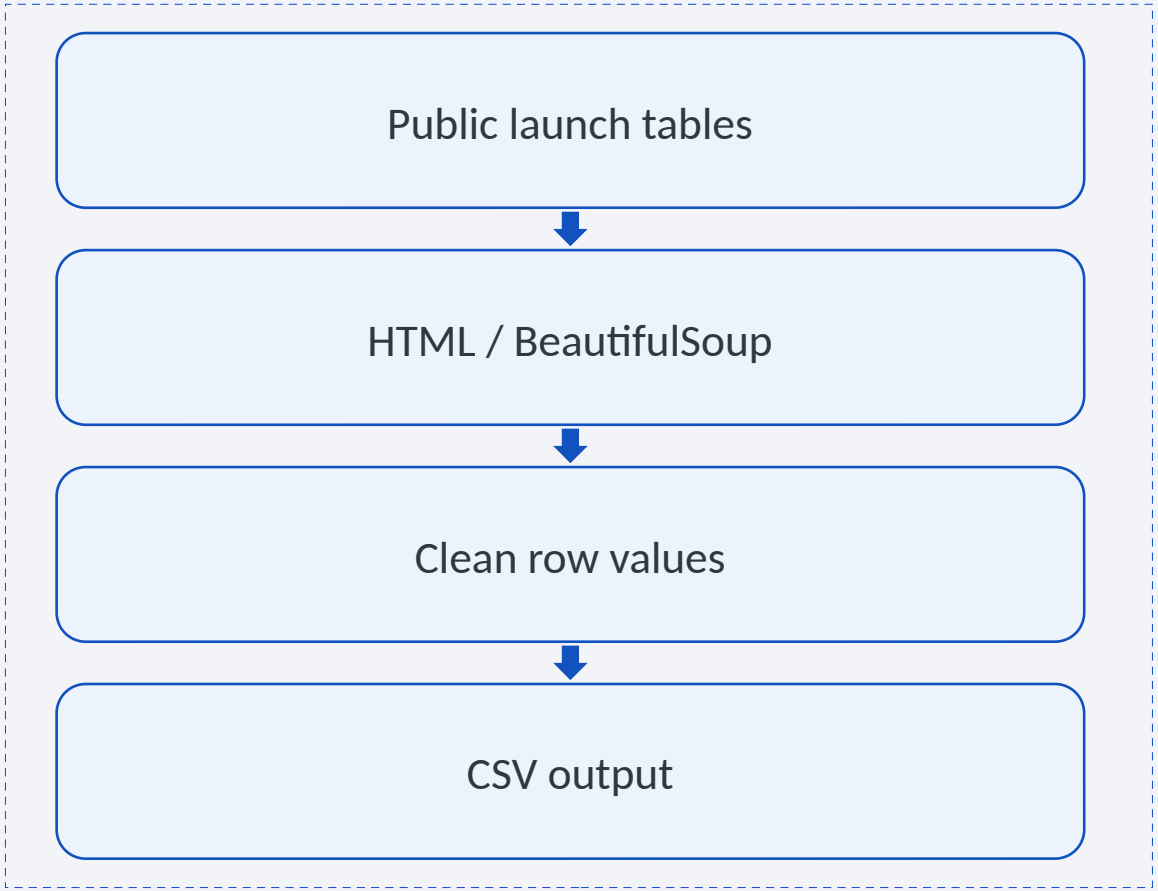
```
response = requests.get(static_url)
soup = BeautifulSoup(response.text, 'html.parser')
html_tables = soup.find_all('table', class_='wikitable')
first_launch_table = html_tables[2]
for th in first_launch_table.find_all("th"):
    column_names.append(extract_column_from_header(th))
```

Observed output / evidence

```
soup.title -> List of Falcon 9 and Falcon Heavy launches
column_names -> ['Flight No.', 'Launch site', 'Payload', ...]
scraped rows exported as CSV-ready records
```

GitHub notebook: Module 1 · Wikipedia scraping

[GitHub notebook link]



Data Wrangling

Extract the first-stage landing outcome.

Map outcomes to binary Class: successful landing = 1; otherwise = 0.

Handle missing, inconsistent and categorical fields.

Notebook evidence · Data wrangling

Code excerpt

```
df = pd.read_csv("dataset_part_1.csv")
landing_outcomes = df['Outcome'].value_counts()
bad_outcomes = set(landing_outcomes.keys()[[1,3,5,6,7]])
landing_class = [0 if outcome in bad_outcomes else 1
                 for outcome in df['Outcome']]
df['Class'] = landing_class
```

Observed output / evidence

```
df[['Class']].head() -> 0, 0, 0, 0, 0 ...
Success rate (Class mean) -> 0.6667
Exported dataset_part_2.csv for EDA and ML
```

GitHub notebook: Module 1 · Data wrangling

Open the source notebook / script on GitHub

EDA with Data Visualization

Compare launch site, payload mass, orbit and flight number.

Analyze yearly landing rates and category counts.

Plots reveal correlations, not causes.

Notebook evidence · Visual EDA

Code excerpt

```
launch_counts = df['LaunchSite'].value_counts()
launch_counts.plot(kind='bar', title='Launches per Site')
orbit_counts = df['Orbit'].value_counts()
landing_outcomes = df['Outcome'].value_counts()
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

Observed output / evidence

```
LaunchSite counts -> CCAFS SLC 40: 55, KSC LC 39A: 22, VAFB SLC
4E: 13
Orbit counts -> GTO: 27, ISS: 21, VLEO: 14, PO: 9, ...
These summaries feed the charts shown in the Results section
```

GitHub notebook: Module 1 · EDA with visualization

Open the source notebook / script on GitHub

EDA with SQL

DISTINCT launch sites and CCAFS record filters.

SUM and AVG payload calculations, earliest ground landing.

Drone ship outcomes, maximum payload and date-ranked results.

Notebook evidence · SQL exploration

Code excerpt

```
%sql SELECT DISTINCT Launch_Site FROM SPACEXTABLE;
%sql SELECT SUM("Payload_Mass__kg_") FROM SPACEXTABLE
     WHERE Customer LIKE '%NASA (CRS)%';
%sql SELECT MIN(Date) FROM SPACEXTABLE
     WHERE Landing_Outcome = 'Success (ground pad)';
```

Observed output / evidence

```
Launch sites -> CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, CCAFS SLC-40
Total NASA CRS payload -> 48213
First successful ground landing -> 2015-12-22
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Build an Interactive Map with Folium

Map launch sites using latitude and longitude.

Color outcomes and add site markers.

Inspect approximate distance to local transport and coastal features.

Notebook evidence · Folium map

Code excerpt

```
site_map = folium.Map(location=coordinate, zoom_start=5)
for launch_site, coordinate in nasa_launch_sites.items():
    folium.Circle([coordinate[0], coordinate[1]],
                  radius=1000, color="#d35400").add_to(site_map)
folium.Marker(coordinate, icon=DivIcon(...)).add_to(site_map)
```

Observed output / evidence

```
Launch site markers and popup labels created successfully
Saved geospatial CSV reused for slide figures 35–37
```

GitHub notebook: Module 3 · Folium map

Open the source notebook / script on GitHub

Build a Dashboard with Plotly Dash

Launch-site dropdown changes success/failure pie chart.

Payload range slider filters a scatter plot.

Color and hover data help compare outcomes and booster versions.

Code evidence · Dash app

Code excerpt

```
spacex_df = pd.read_csv("spacex_launch_dash.csv")
app.layout = html.Div([dcc.Dropdown(id="site-dropdown", ...),
                       dcc.RangeSlider(id="payload-slider", ...),
                       dcc.Graph(id="success-pie-chart"),
                       dcc.Graph(id="success-payload-scatter-chart")])
@app.callback(...)
def update_charts(selected_site, payload_range): ...
```

Observed output / evidence

```
Interactive filters drive the pie chart and scatter plot
Dashboard outputs are shown again on slides 39–41
```

GitHub script: Module 3 · spacex_dash_app.py

Open the source notebook / script on GitHub

Predictive Analysis (Classification)

Encode predictors and split into training and held-out test sets.

Tune Logistic Regression, SVM, Decision Tree and KNN by cross-validation.

Compare both test accuracy and confusion matrix errors.

Notebook evidence · Classification workflow

Code excerpt

```
transform = preprocessing.StandardScaler()
X = transform.fit_transform(X)
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=2)
logreg_cv = GridSearchCV(LogisticRegression(), parameters, cv=10)
accuracy_lr = logreg_cv.score(X_test, Y_test)
```

Observed output / evidence

```
Y_test.shape -> (18,)
Best CV scores observed for LR, SVM, Decision Tree and KNN
All four models reached test accuracy = 0.8333
```

GitHub notebook: Module 4 · ML prediction

Open the source notebook / script on GitHub

Results

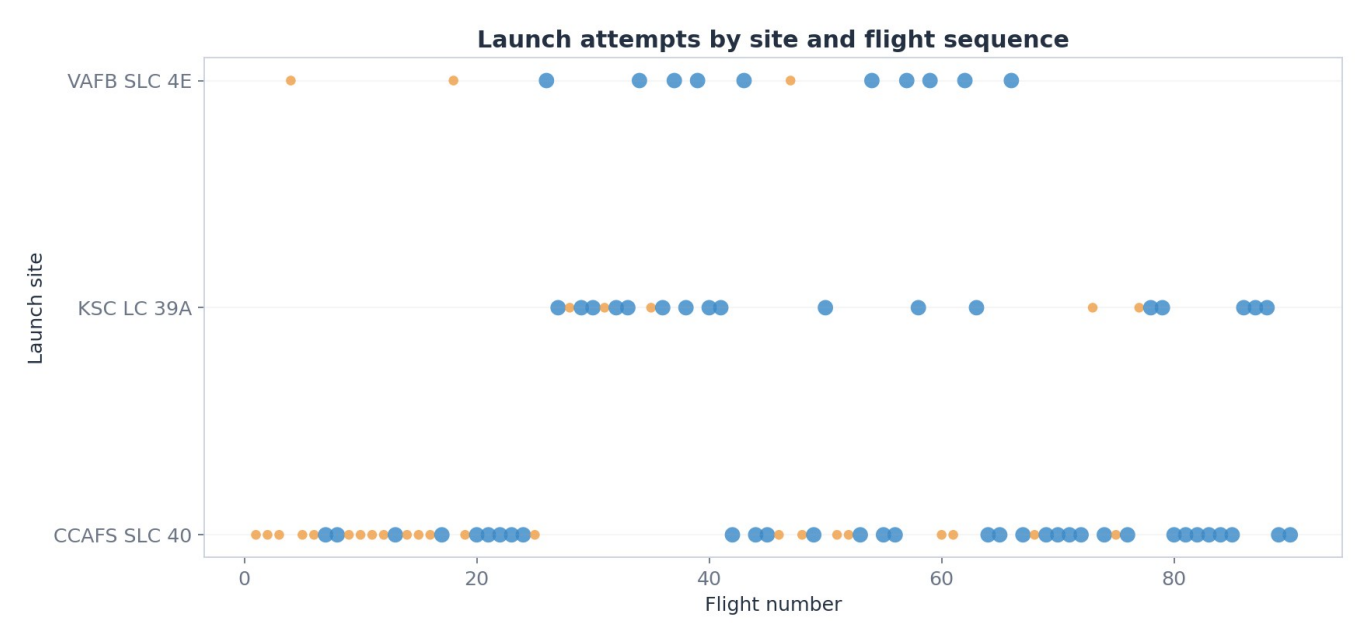
- EDA: launch history, orbit and payload patterns.
- Interactive analysis: geographic launch sites and dashboard filters.
- Predictive modeling: four classifiers on the same holdout sample.

Section 2

Insights drawn from EDA

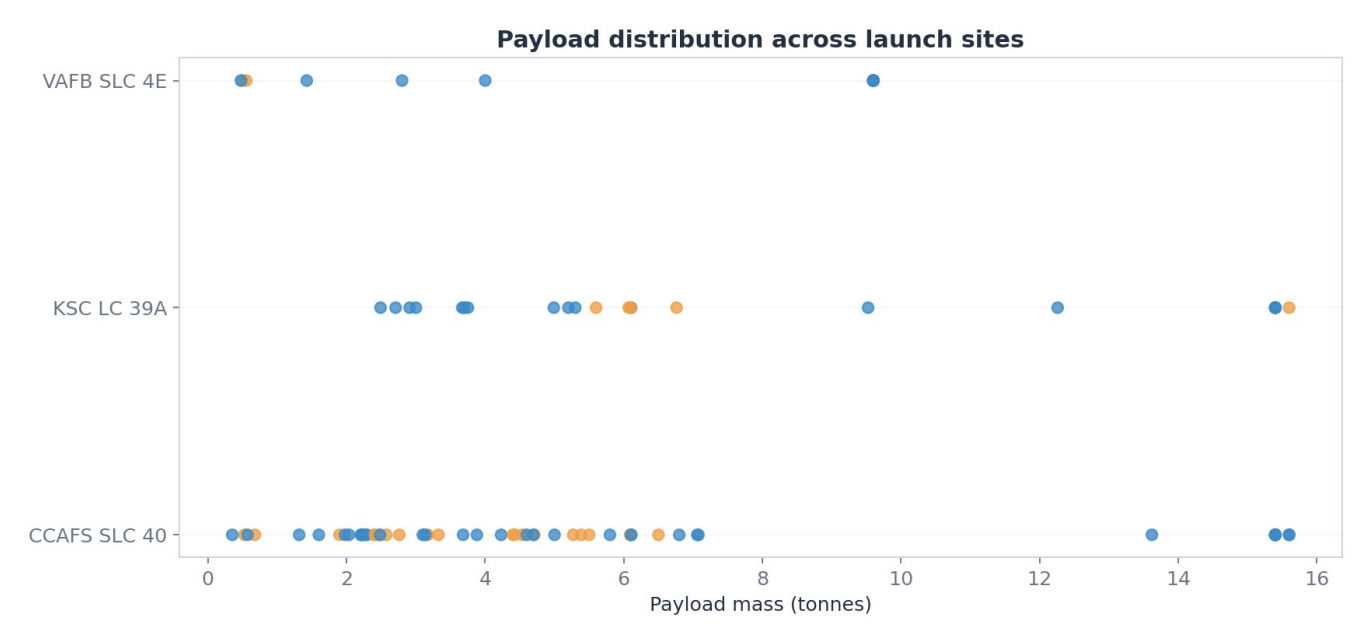
Flight Number vs. Launch Site

- Later flights contain more successful landings. Site histories differ.
- Visualization from course-provided 90-row modeling dataset.



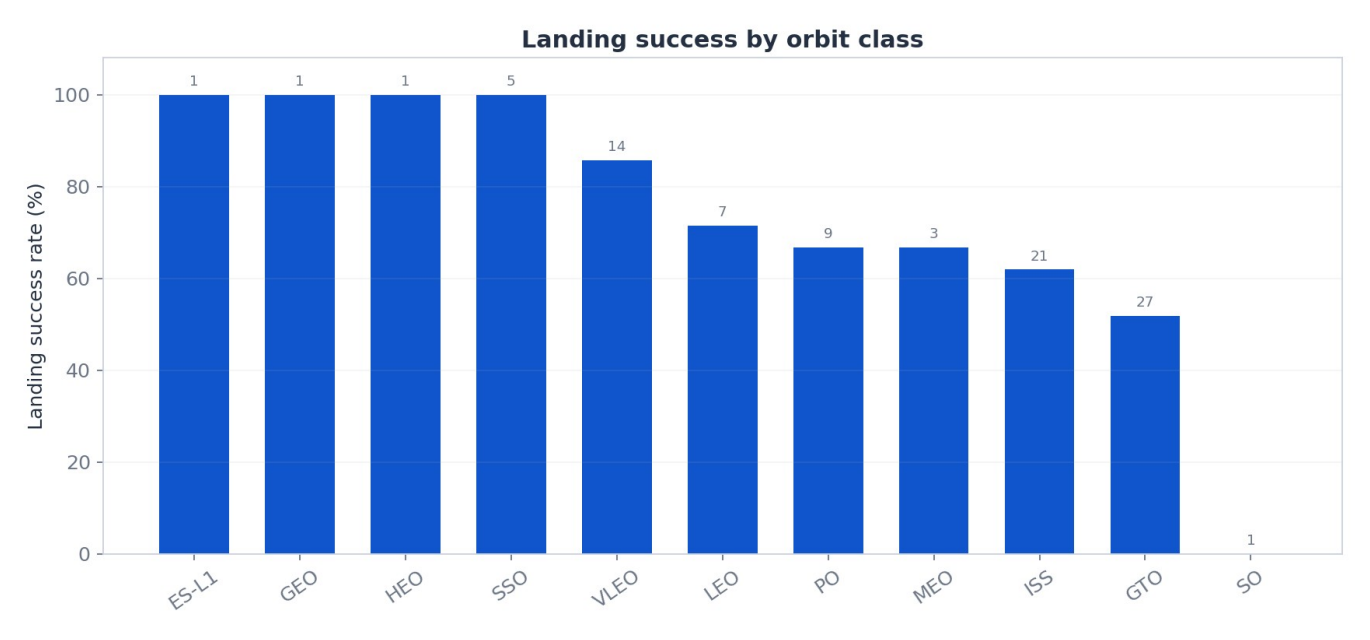
Payload vs. Launch Site

- Payload masses overlap between successful and unsuccessful landings.
- Visualization from course-provided 90-row modeling dataset.



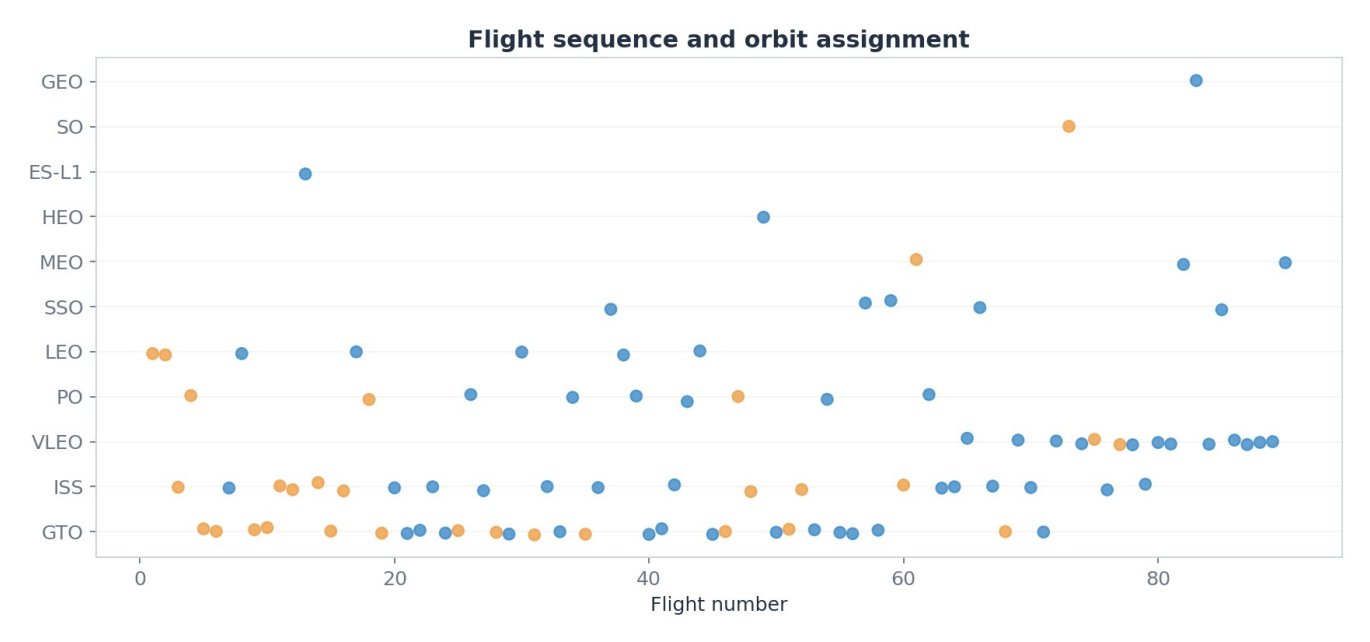
Success Rate vs. Orbit Type

- Landing success varies by orbit category. Small group counts matter.
- Visualization from course-provided 90-row modeling dataset.



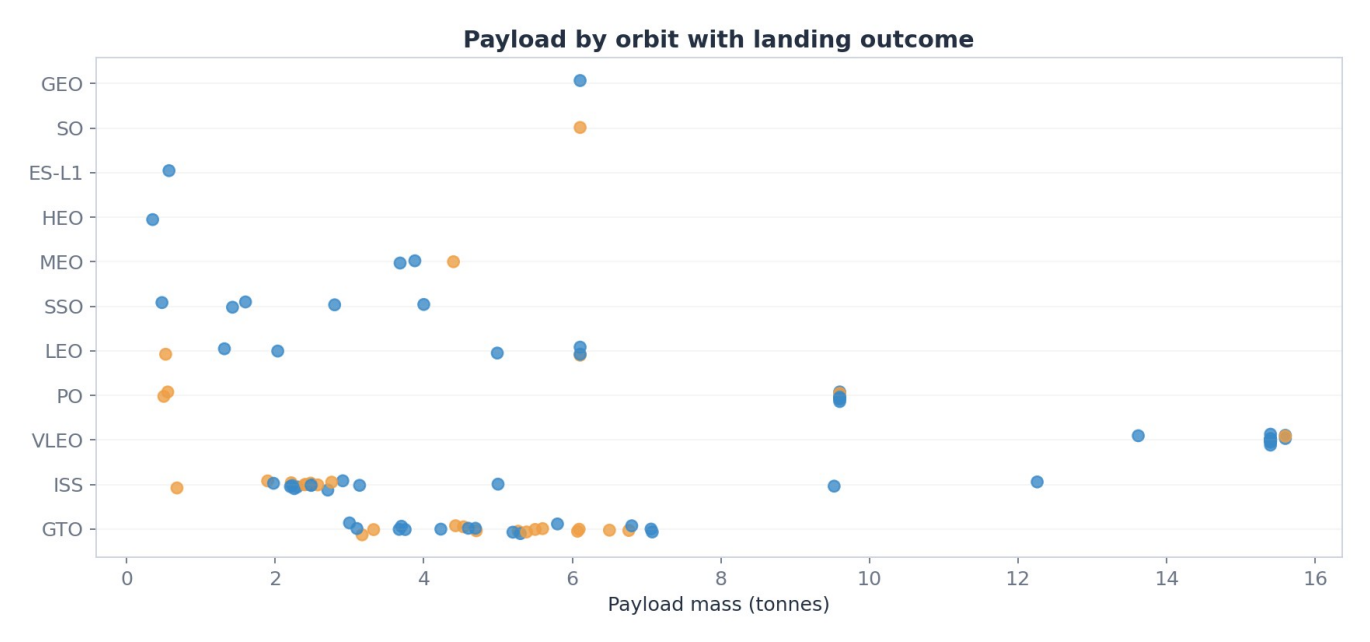
Flight Number vs. Orbit Type

- Orbit profiles change across the historical launch sequence.
- Visualization from course-provided 90-row modeling dataset.



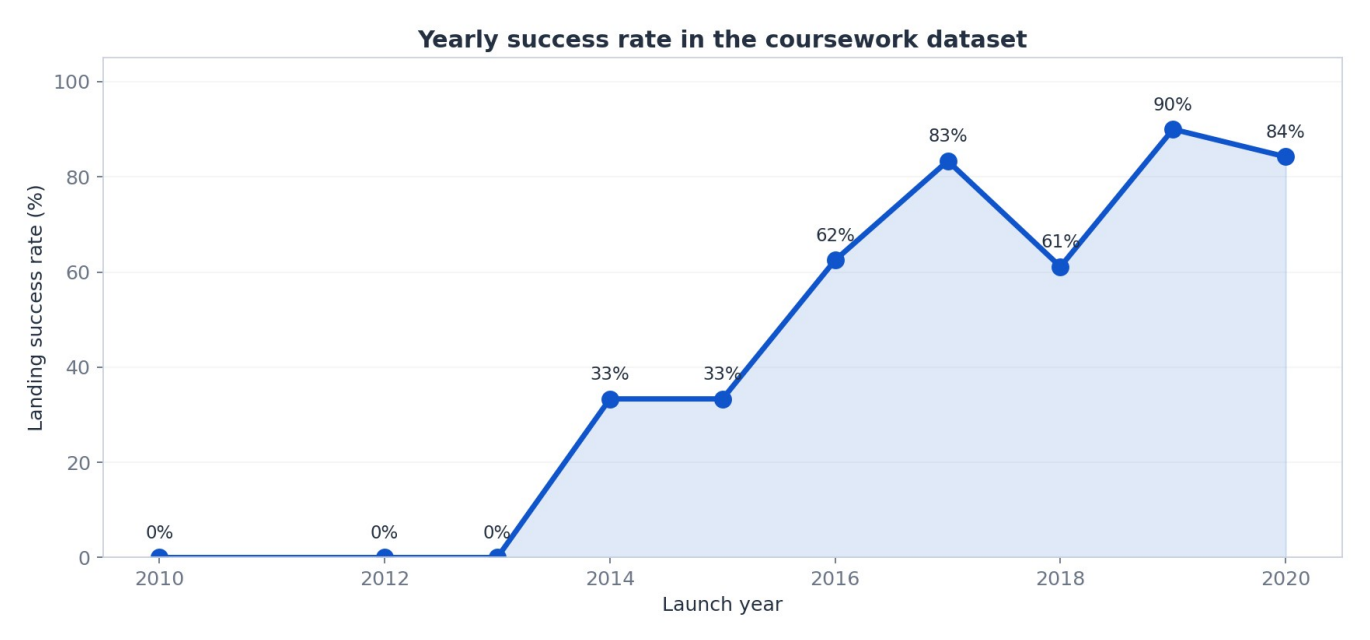
Payload vs. Orbit Type

- Payload ranges differ by mission orbit; outcomes still overlap.
- Visualization from course-provided 90-row modeling dataset.



Launch Success Yearly Trend

- The sample shows an improving landing trend across launch years.
- Visualization from course-provided 90-row modeling dataset.



All Launch Site Names

Unique launch-site strings in the SQL coursework data:

CCAFS LC-40 | VAFB SLC-4E

KSC LC-39A | CCAFS SLC-40

Distinct strings may include naming variants.

SQL evidence · Slide 24

Code excerpt

```
%sql SELECT DISTINCT Launch_Site FROM SPACEXTABLE;
```

Observed output / evidence

```
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Launch Site Names Begin with 'CCA'

Query: Launch_Site LIKE 'CCA%' LIMIT 5

2010-06-04	CCAFS LC-40	F9 v1.0 B0003
2010-12-08	CCAFS LC-40	F9 v1.0 B0004
2012-05-22	CCAFS LC-40	F9 v1.0 B0005
2012-10-08	CCAFS LC-40	F9 v1.0 B0006
2013-03-01	CCAFS LC-40	F9 v1.0 B0007

SQL evidence · Slide 25

Code excerpt

```
%sql SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5;
```

Observed output / evidence

```
2010-06-04  CCAFS LC-40  F9 v1.0 B0003
2010-12-08  CCAFS LC-40  F9 v1.0 B0004
2012-05-22  CCAFS LC-40  F9 v1.0 B0005
2012-10-08  CCAFS LC-40  F9 v1.0 B0006
2013-03-01  CCAFS LC-40  F9 v1.0 B0007
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Total Payload Mass

48,213 kg

Total payload for rows with customer matching NASA (CRS).

SQL: SUM(Payload_Mass__kg_) WHERE Customer LIKE '%NASA (CRS)%'.

Source: recorded course SQL notebook output.

SQL evidence · Slide 26

Code excerpt

```
%sql SELECT SUM("Payload_Mass__kg_") FROM SPACEXTABLE
     WHERE Customer LIKE '%NASA (CRS)%';
```

Observed output / evidence

```
Total_Payload = 48213
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Average Payload Mass by F9 v1.1

2,928.4 kg

Notebook output for AVG(Payload_Mass__kg_) using Booster_Version = 'F9 v1.1'.

This is the normalized SQL notebook result; raw CSV labels can differ.

SQL evidence · Slide 27

Code excerpt

```
%sql SELECT AVG("Payload_Mass__kg_") FROM SPACEXTABLE
     WHERE Booster_Version = 'F9 v1.1';
```

Observed output / evidence

```
Avg_Payload = 2928.4
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

First Successful Ground Landing Date

22 December 2015

Earliest successful ground pad landing in the SQL records.

Query: MIN(Date) where Landing_Outcome = 'Success (ground pad)'.

SQL evidence · Slide 28

Code excerpt

```
%sql SELECT MIN(Date) FROM SPACEXTABLE
     WHERE Landing_Outcome = 'Success (ground pad)';
```

Observed output / evidence

```
First_Successful_GroundPad_Landing = 2015-12-22
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Successful Drone Ship Landing with Payload between 4000 and 6000

Successful drone ship outcomes with payload strictly between 4,000 and 6,000 kg:

F9 FT B1022 | 4,696 kg

F9 FT B1026 | 4,600 kg

F9 FT B1021.2 | 5,300 kg

F9 FT B1031.2 | 5,200 kg

SQL evidence · Slide 29

Code excerpt

```
%sql SELECT Booster_Version, Payload_Mass__kg_ FROM SPACEXTABLE
     WHERE Landing_Outcome = 'Success (drone ship)'
       AND Payload_Mass__kg_ BETWEEN 4000 AND 6000;
```

Observed output / evidence

```
F9 FT B1022      4696
F9 FT B1026      4600
F9 FT  B1021.2   5300
F9 FT  B1031.2   5200
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Total Number of Successful and Failure Mission Outcomes

Landing outcomes are not binary in the historic SQL table.

Examples: Success, Success (drone ship), Failure (drone ship),

Success (ground pad), No attempt and Controlled (ocean).

The classifier later uses a separate binary landing label.

SQL evidence · Slide 30

Code excerpt

```
%sql SELECT Landing_Outcome, COUNT(*) FROM SPACEXTABLE
     GROUP BY Landing_Outcome;
```

Observed output / evidence

```
Success: 38   |   Success (drone ship): 14
Success (ground pad): 9
Failure (drone ship): 5   |   Controlled (ocean): 5
No attempt: 21 (+ 1 trailing-space variant)
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Boosters Carried Maximum Payload

Maximum payload in the saved SQL source: 15,600 kg.

Examples of tied booster records:

F9 B5 B1048.4 | F9 B5 B1049.4 | F9 B5 B1051.3

F9 B5 B1056.4 | F9 B5 B1048.5

SQL returns additional boosters tied at this maximum.

SQL evidence · Slide 31

Code excerpt

```
%sql SELECT DISTINCT Booster_Version FROM SPACEXTABLE
     WHERE Payload_Mass__kg_ = (SELECT MAX(Payload_Mass__kg_)
                                FROM SPACEXTABLE);
```

Observed output / evidence

```
Max payload = 15600 kg
Example matches: F9 B5 B1048.4, B1049.4, B1051.3,
B1056.4, B1048.5, ...
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

2015 Launch Records

2015 failed drone ship landings in the notebook:

January | F9 v1.1 B1012 | CCAFS LC-40

April | F9 v1.1 B1015 | CCAFS LC-40

Two matching records.

SQL evidence · Slide 32

Code excerpt

```
%sql SELECT substr(Date,6,2) AS Month, Landing_Outcome,
     Booster_Version, Launch_Site FROM SPACEXTABLE
     WHERE Landing_Outcome = 'Failure (drone ship)'
       AND substr(Date,0,5) = '2015';
```

Observed output / evidence

```
01  Failure (drone ship)  F9 v1.1 B1012  CCAFS LC-40
04  Failure (drone ship)  F9 v1.1 B1015  CCAFS LC-40
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

No attempt: 10 | Success (drone ship): 5

Failure (drone ship): 5 | Success (ground pad): 3

Controlled (ocean): 3 | Uncontrolled (ocean): 2

Failure (parachute): 2 | Precluded (drone ship): 1

Date filter: 04 Jun 2010 through 20 Mar 2017.

SQL evidence · Slide 33

Code excerpt

```
%sql SELECT Landing_Outcome, COUNT(*) FROM SPACEXTABLE
     WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
     GROUP BY Landing_Outcome ORDER BY Outcome_Count DESC;
```

Observed output / evidence

```
No attempt: 10
Success (drone ship): 5    Failure (drone ship): 5
Success (ground pad): 3    Controlled (ocean): 3
Uncontrolled (ocean): 2    Failure (parachute): 2
Precluded (drone ship): 1
```

GitHub notebook: Module 2 · SQL notebook

Open the source notebook / script on GitHub

Section 3

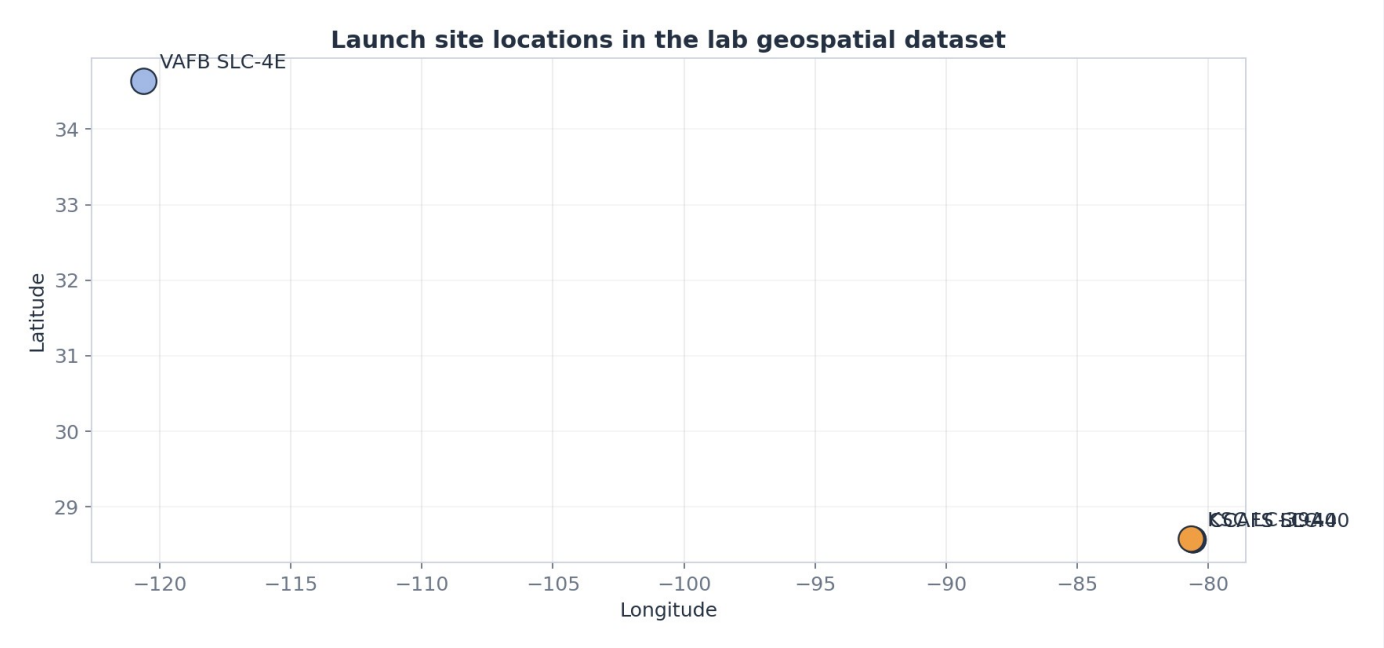
Launch Sites Proximities Analysis



Launch Site Locations

Saved geospatial coordinates show launch-site locations.

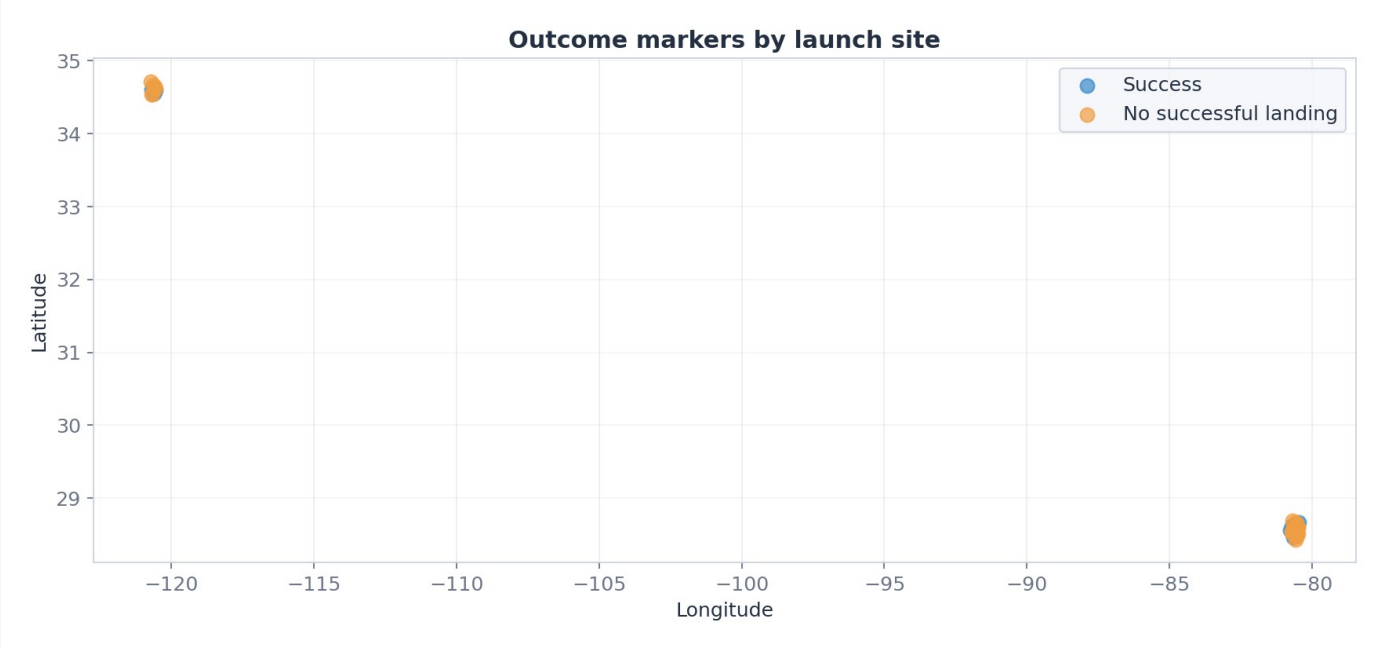
Chart reconstructed from lab CSV, not a live Folium screenshot.



Landing Outcomes by Site

Success and failure outcomes are color-coded by site.

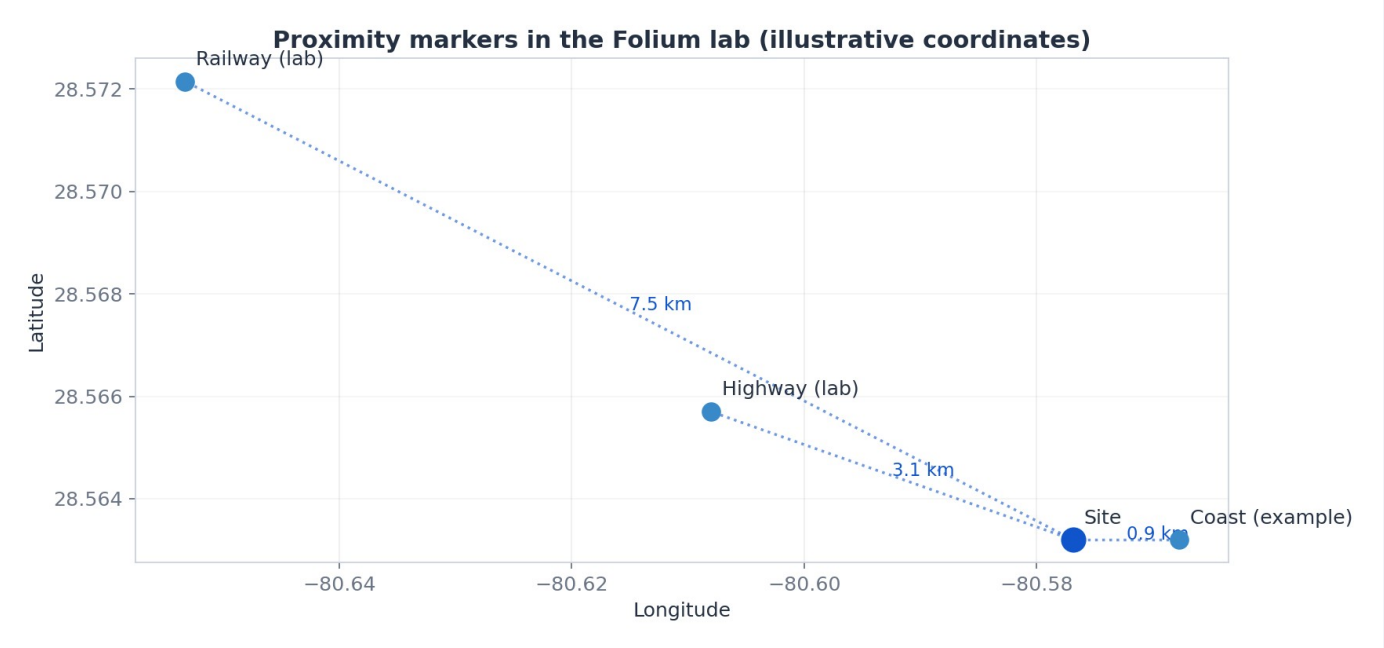
Reconstructed plot from the saved geospatial dataset.

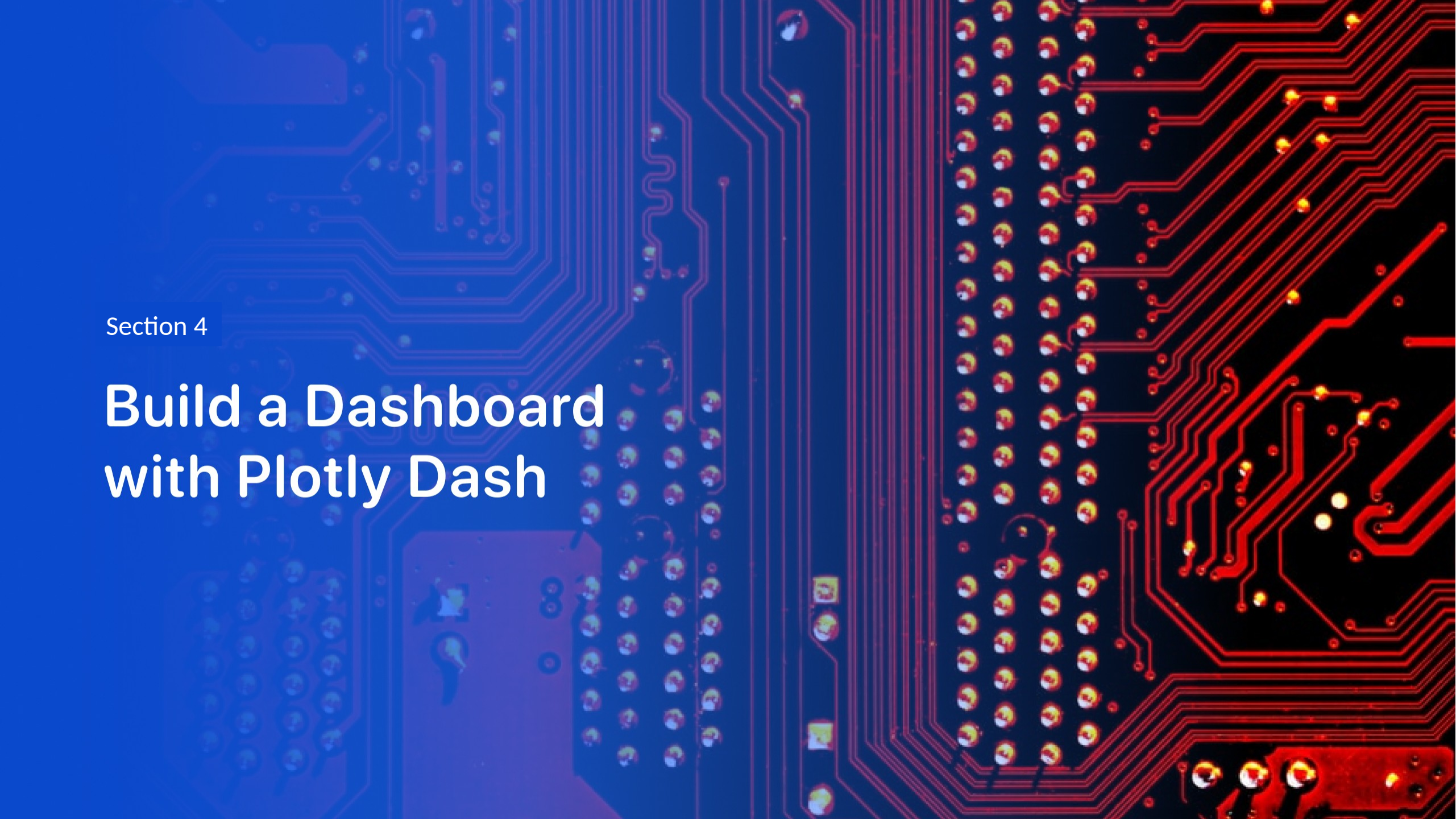


Launch Site Proximity

Illustrative coast, railway and road proximity measurements.

Approximate distances, not surveyed geographic distances.

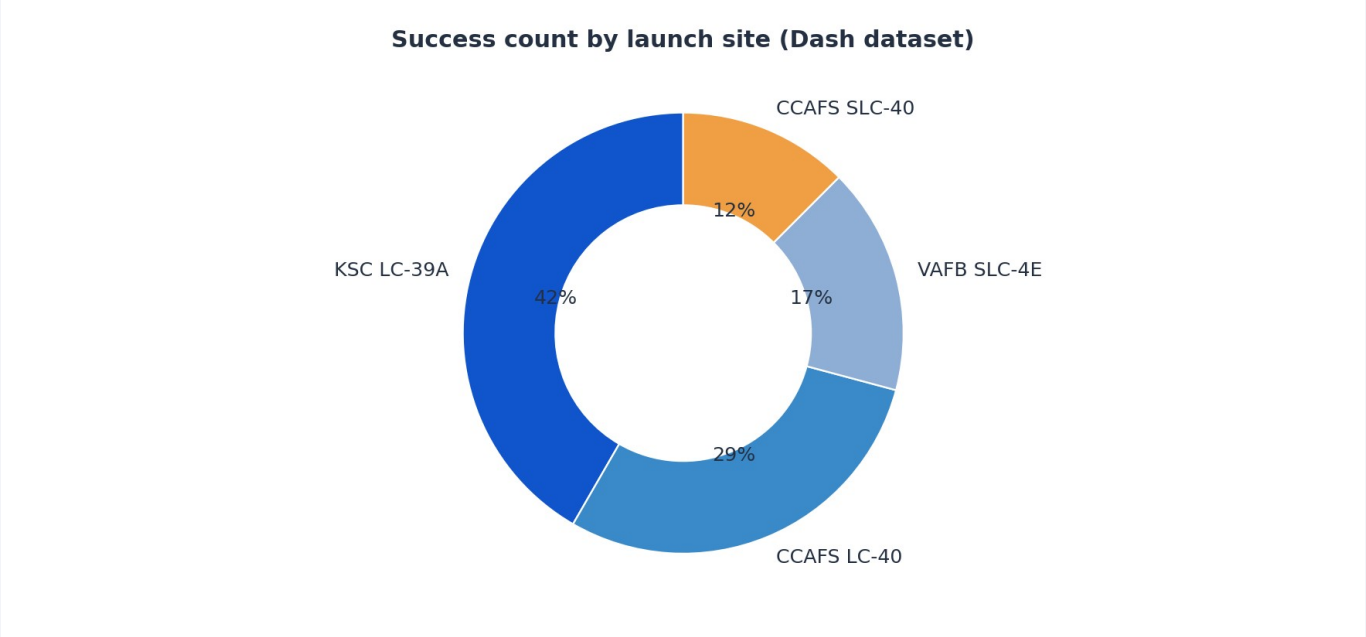



Section 4
Build a Dashboard with Plotly Dash

Landing Success Count by Site

Successful landings are grouped by launch site.

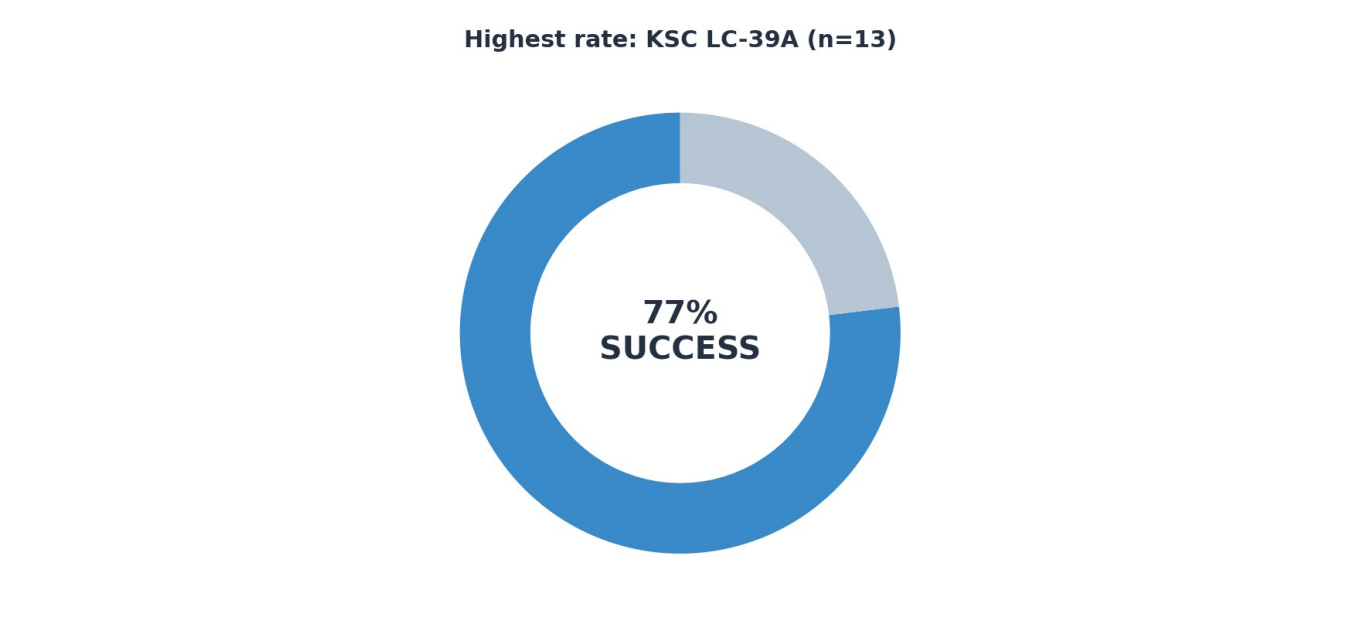
Slices show share of successes, not each site success rate.



Highest Site Success Ratio

Per-site ratio: successful landings / total attempts.

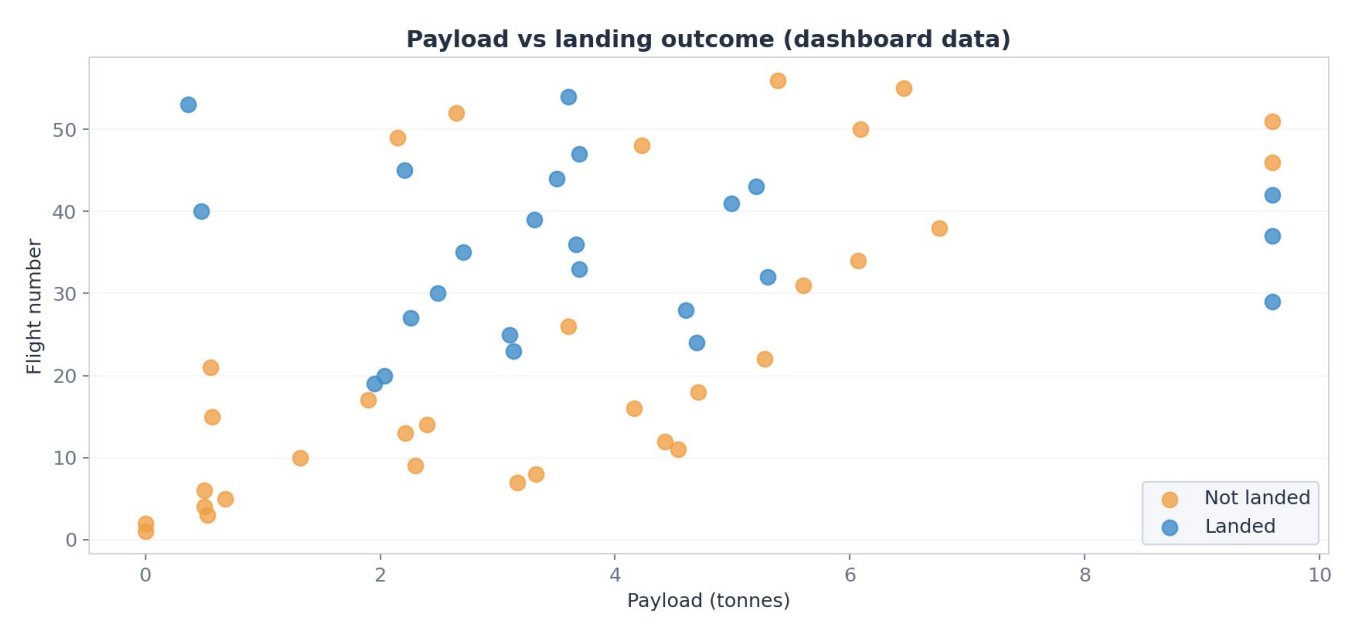
Reconstructed from 56-record Dash dataset.



Payload and Launch Outcome

Scatter plot separates success vs. failure.

Original app provides site and payload-range filters.



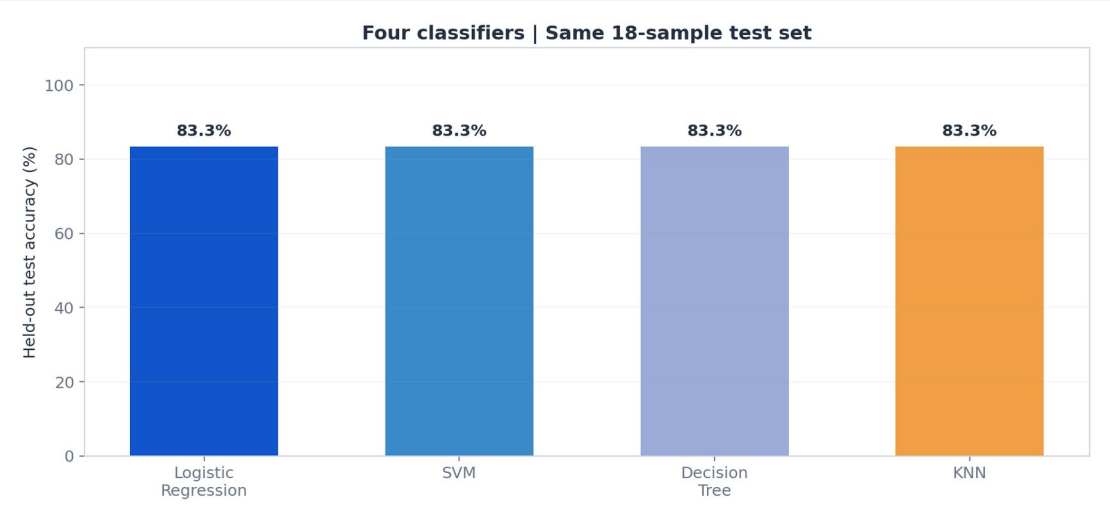
Section 5

Predictive Analysis (Classification)



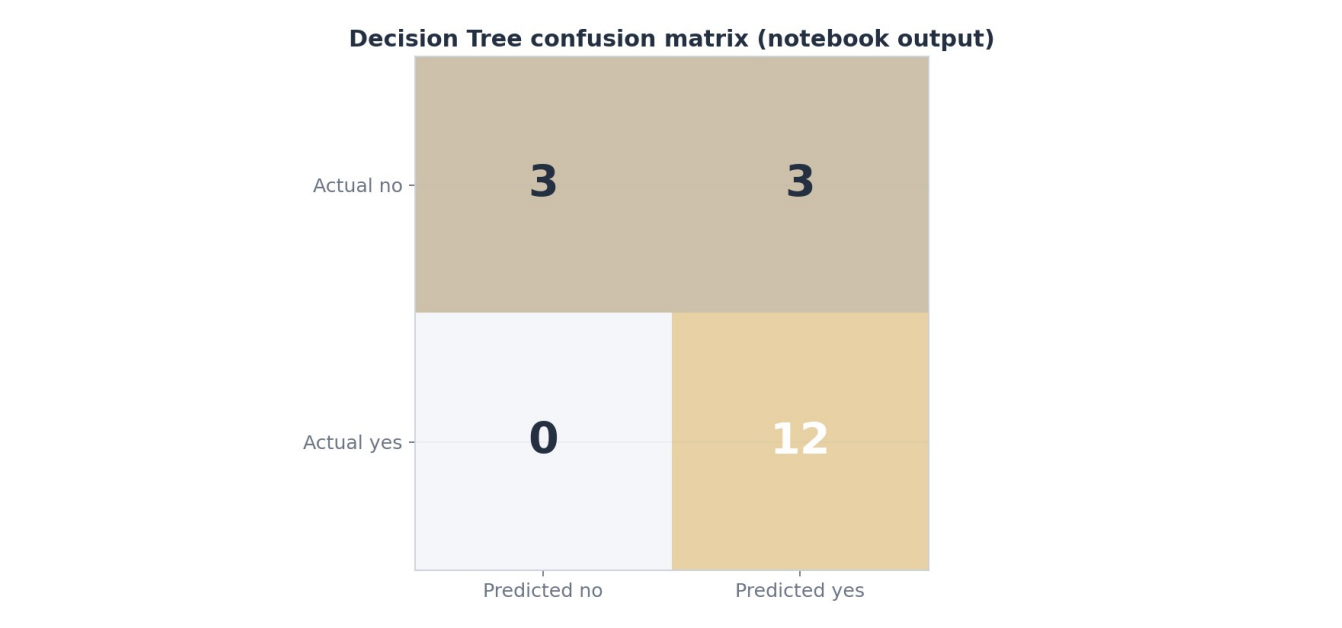
Classification Accuracy

- Test split: 18 launch records.
- Logistic Regression, SVM, Decision Tree and KNN: 83.3% accuracy.
- Decision Tree leads cross-validation, but tied on held-out data.



Confusion Matrix

- Decision Tree confusion matrix from notebook:
- TN = 3 FP = 3
- FN = 0 TP = 12
- Accuracy = 15 / 18 = 83.3%.
- Positive recall = 100%; specificity = 50%.



Conclusions

Historical landing success improves with flight sequence in these records.

Landing outcome also varies by orbit, mission and launch site.

All four models tie on a small 18-sample test set.

Interpret accuracy with cross-validation and confusion matrix.

This educational analysis is not a live launch-risk model.

Appendix

Source notebooks and reproducibility:

Module 1: SpaceX REST API, web scraping and data wrangling.

Module 2: SQL data exploration.

Module 3: Folium mapping and Plotly Dash dashboard.

Module 4: model tuning and confusion matrix.

GitHub: github.com/kgeckin/IBM_Applied_Data_Science_Capstone

Thank you!

